SOC Analyst Lab

Introduction

This project involves analyzing log data to determine what happened to a compromised web server honeypot. Essentially, it is a Capture the Flag (CTF) challenge. The files from the compromised web server were sourced from cyberdefenders.org[1] under the lab titled "Hammered Lab."

Objectives

Here is a summary of the objectives for this lab

- Demonstrating the ability to deploy virtual machines and install operating systems
- Showcasing the use of Linux and command line.
- Incident response and detection
- Log analysis
- Threat Detection

Setup

In order to analyze the logs, a lab is required to be built. This will involve creating a virtual machine using VirtualBox, running Kali Linux on macOS platform.

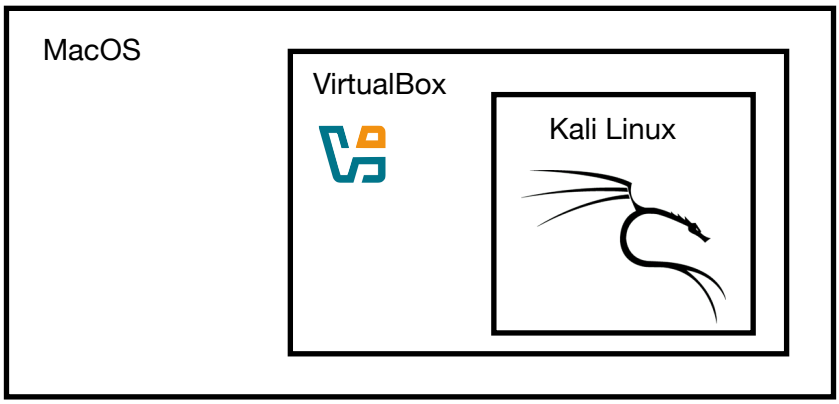

Figure 1: Visual Representation of the lab

[1] https://cyberdefenders.org/blueteam-ctf-challenges/hammered/

The software that will be used in this lab

- VirtualBox 7.1.4
- Kali 2024.4
- macOS Sequoia

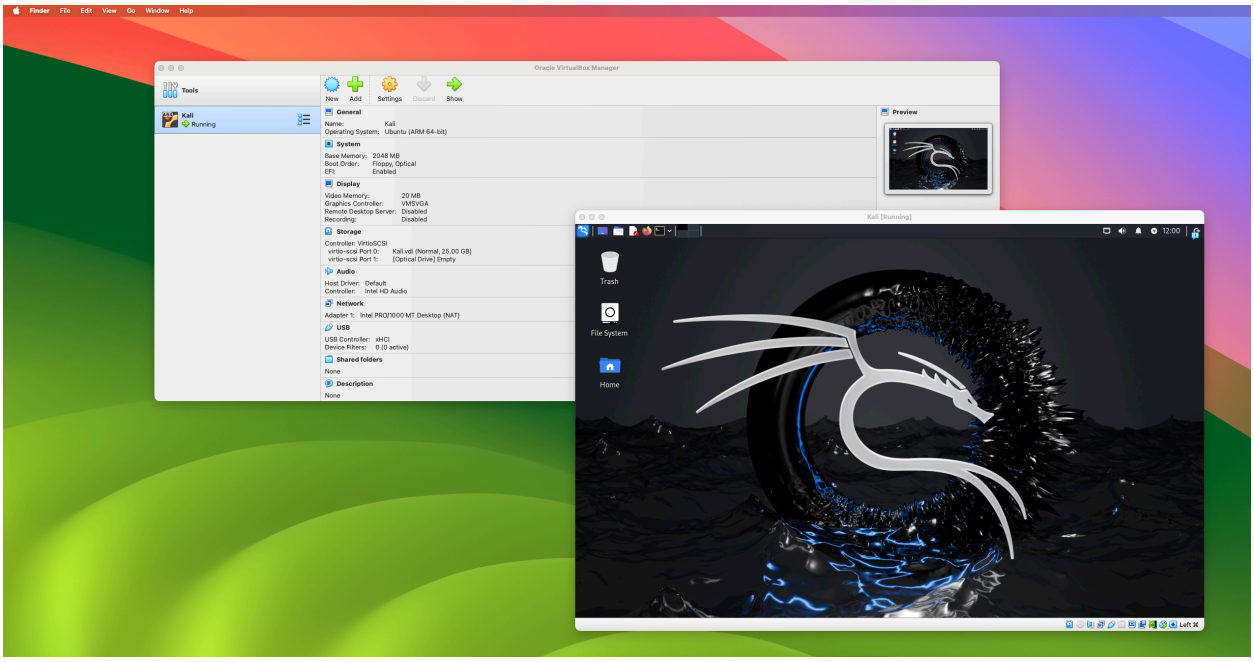

Figure 2: Screenshot of the environment

Next step is to download and extract the files from cyberdefenders.org.

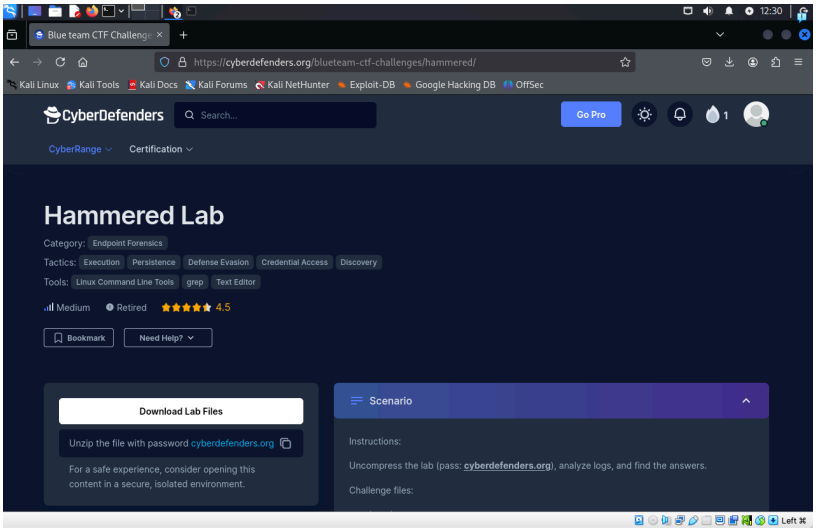

Figure 3: Cyberdefenders website

Using the terminal can confirm that the files have been downloaded and extracted.

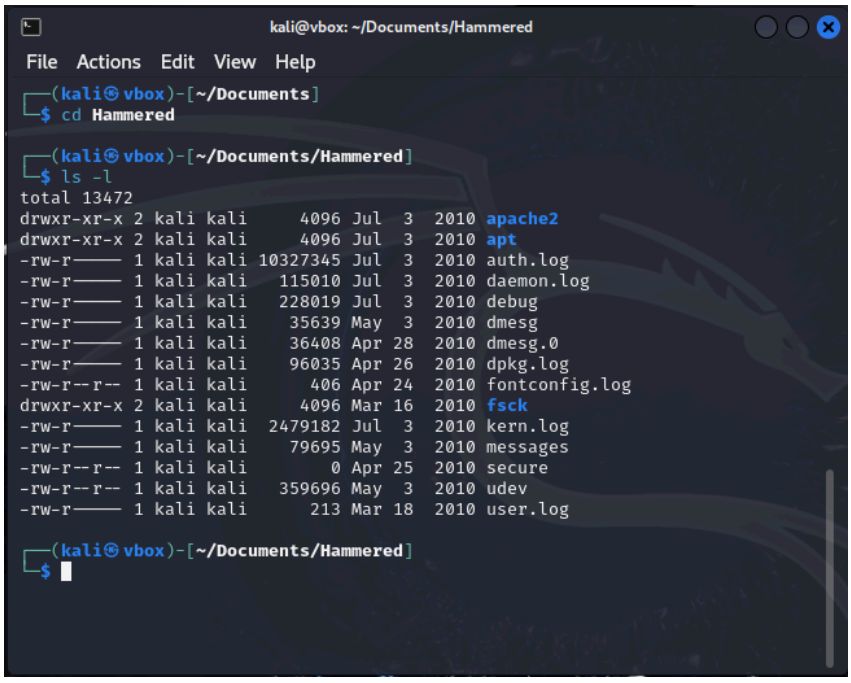

Figure 4: List of files for the lab

The setup is complete and now to begin the investigation.

Investigation

To begin the investigation, I will use the website challenge questions as a guide to help discovering whether or not the honeypot server has been compromised. To start with though, looking through the files in the Hammered directory, there are 5 files of interest.

- kern.log - Contain system messages from the Linux kernel
- auth.log - Keeps track of authentication and security related events
- daemon.log - Records system service and background process events
- dmesg - Displays kernel ring buffer messages and hardware events
- apache2 - Record web server access and error details

What service did the attackers use to gain access to the system?

We can refer to the auth.log to check for any failed or successful login attempts. This should show what the attacker was trying to log into.

The screenshot below shows a snapshot of the auth.log. There are multiple authentication failures using the SSH service.

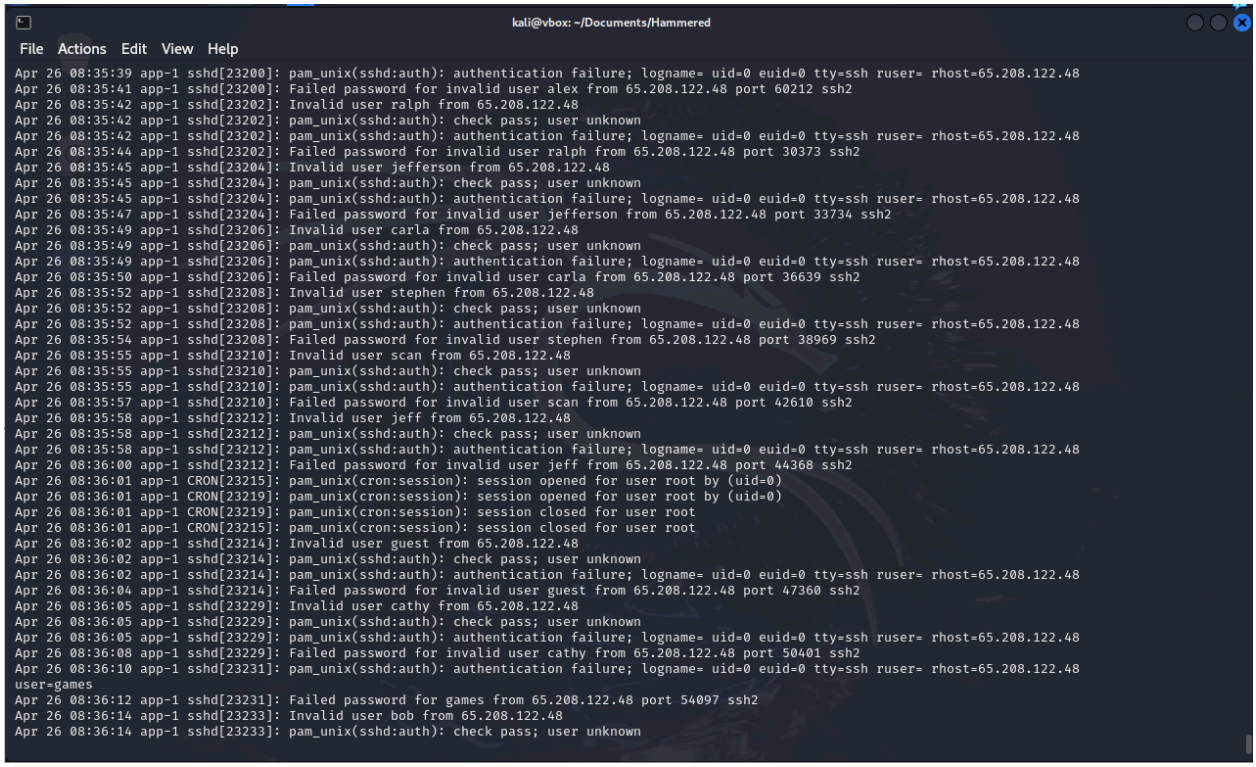

Figure 5: Snapshot of auth.log

The sheer number of failed logins, in a relatively short space of time is an indicator of comprise for a brute force attempt.

The service being attacked is SSH.

What is the operating system version of the targeted system?

The dmesg logs contains kernel ring buffer data. This data can provide insights into the system startup, hardware failures and kernel issues. In this case, we can output the beginning of the file to show what operating system it is running.

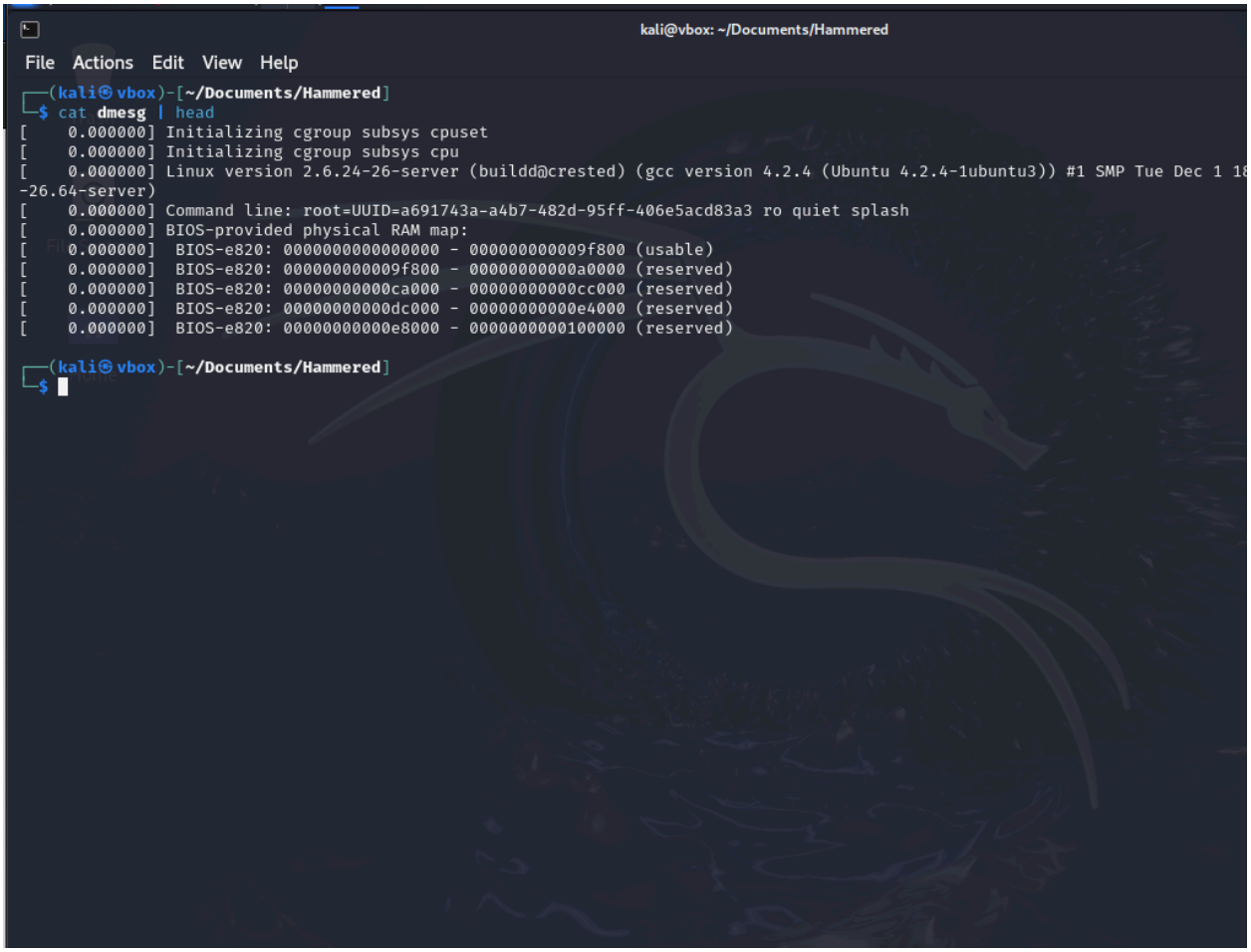

Figure 6: Output of first 10 lines of dmesg log

The operating system in question is: 4.2.4-1ubuntu3.

Q3: What is the name of the compromised account?

The auth.log used previously its likely to contain the answer. From an attackers perspective, they will seek to compromise the account with the highest privileges. My assumption is an administrative account of some form. Using the grep command can search for occurrences of the word ‘admin’ and ‘root’. Both of these would indicate the attempts made into the system.

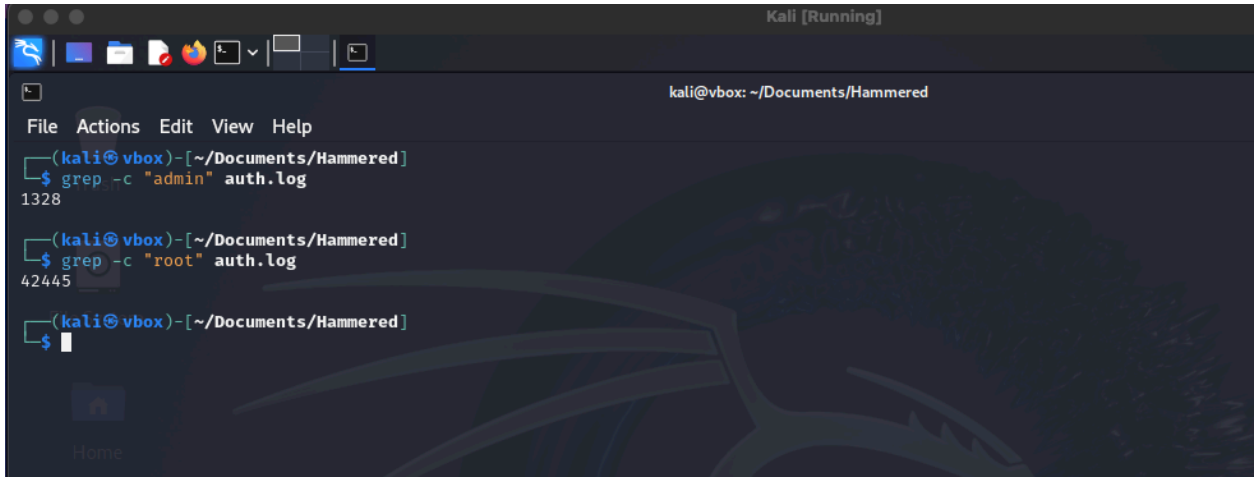

Figure 7: Using grep to count occurrences of a word

From the results of the grep command, the attacker was after the root account given the number of occurrences, 42445 vs 1328 for ‘admin’ in the auth.log file.

The name of the compromised account is ‘root’.

Q4: How many attackers, represented by unique IP addresses, were able to successful access the system after initial failed attempts?

Using the auth.log again, we can run a grep command, looking for the string ‘accepted password’ and ‘root’, this would indicate a successful login. Can also use the awk command to retrieve the fields that are of interest. In this case, it will be the IP addresses.

The following command will be run.

cat auth.log | grep -i “accepted password” | grep root | awk ‘{print $11}’ | uniq

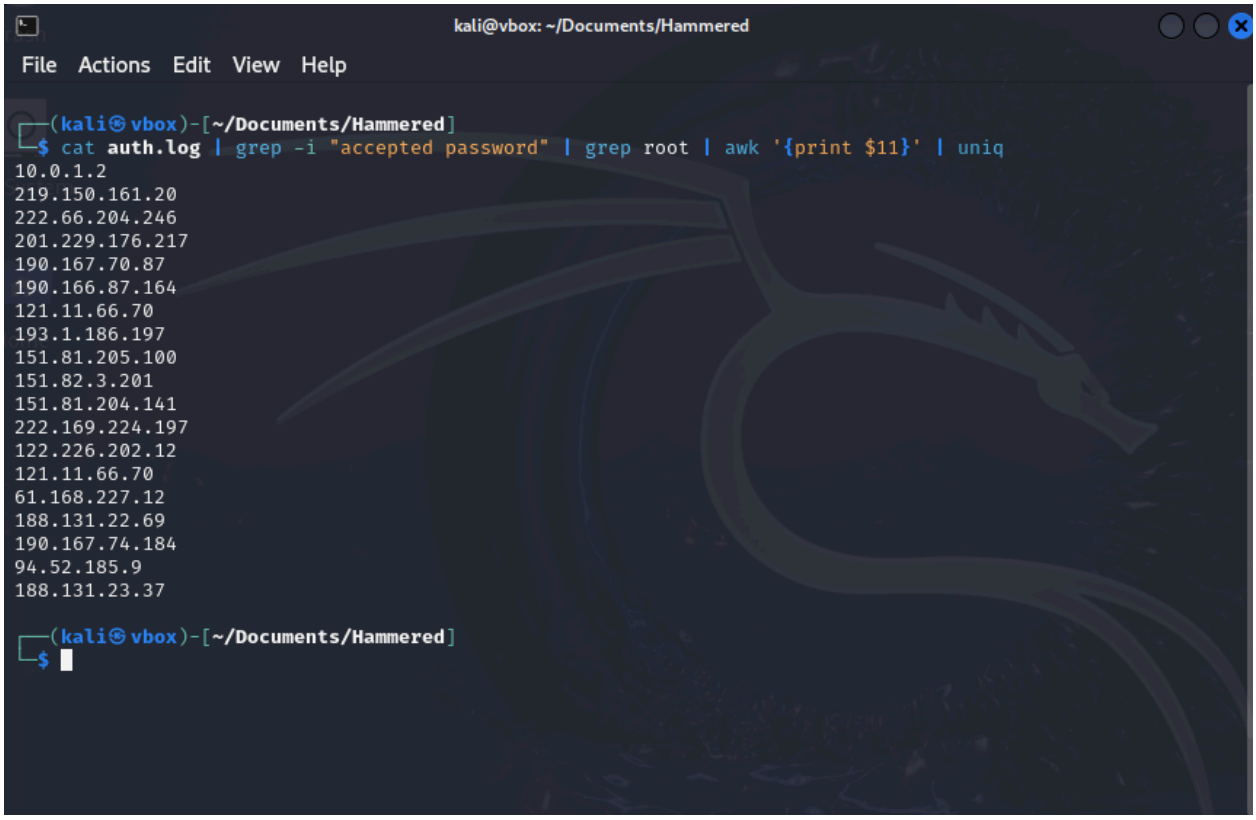

Figure 8: Out of grep command

However, not all the IP addresses above are from the attacker. We can filter using ‘failed password ‘ term with the IP addresses from figure 8.

The wc -l in the command below will count the number of lines that match the grep patterns. In this case, its failed password, root and the IP addresses. A high number of of matches for an IP address will indicate a brute force attempt and thus highly likely from the attacker.

cat auth.log | grep - “failed password” | grep root | grep “IP Address” | wc -l

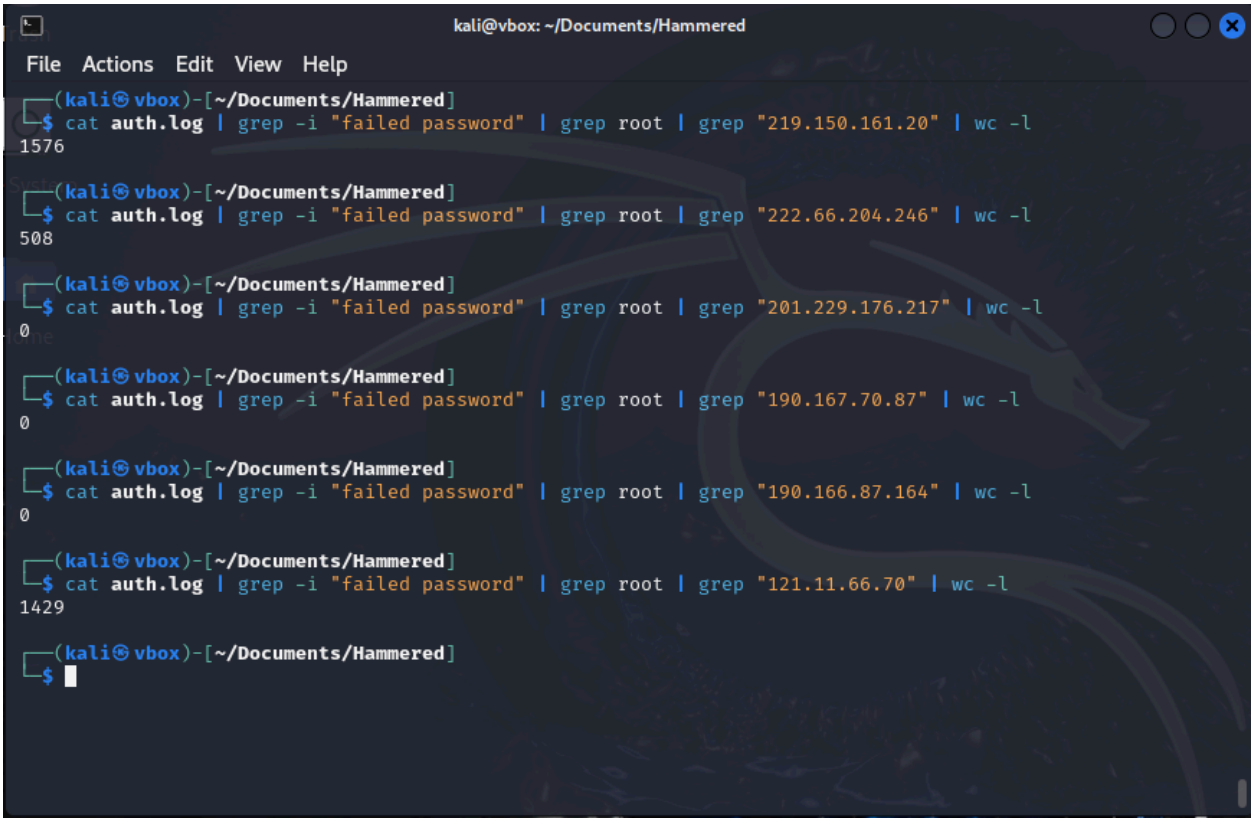

Figure 9: Sample of each Command

The figure above is a snapshot of the command being run for the IP addresses. Not all IP addresses are shown. So far IP address 219.150.161.20, 222.66.204.246 and 121.11.66.70 are from the attacker given the high number of login attempts, ranging from 508 to 1576.

Referring back the figure 8, out of the 18 IP addresses, 6 have been identified as being from the attacker.

219.150.161.20
222.66.204.246
121.11.66.70
222.169.224.197
122.226.202.12
61.168.227.12

Figure 10: IP Addresses from Attacker

There were 6 attackers with unique IP Addresses

Q5: Which attackers IP address successfully logged into the system the most number of times?

In question 4, we identified the number of times each IP address tried to login to the system. The command that produced the highest count is the answer.

The IP address 219.150.161.20 is the attacker that successfully logged into the system.

Q6: How many requests were sent to the Apache Server

Turn our focus to the 'apache2' directory. The contents of the file 'www-access.log' will contain login entries. By counting the number of entries or lines in this file will determine how many requests were sent to the server. The following command will be used.

cat www-access.log | wc -l

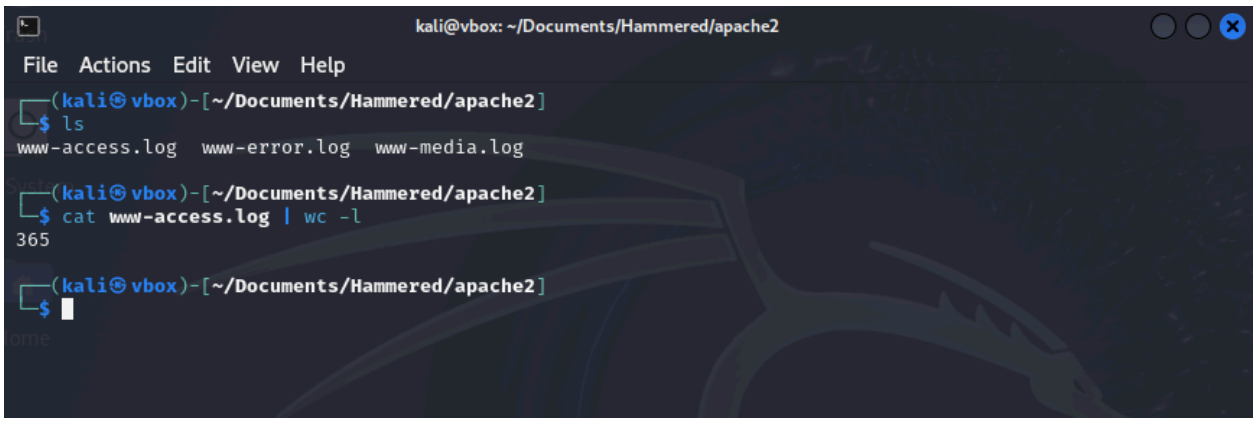

Figure 11: Output number of lines in www-access.log file

Therefore 365 requests were sent to the Apache Server.

Q7: How many rules have been added to the firewall?

Within Linux, the ‘iptables’ are used to manage firewall rules and network traffic. In this scenario, the iptables are located in the auth.log. Simply output the contents of the file with lines matching the pattern ‘iptables’.

cat auth.log | grep iptables

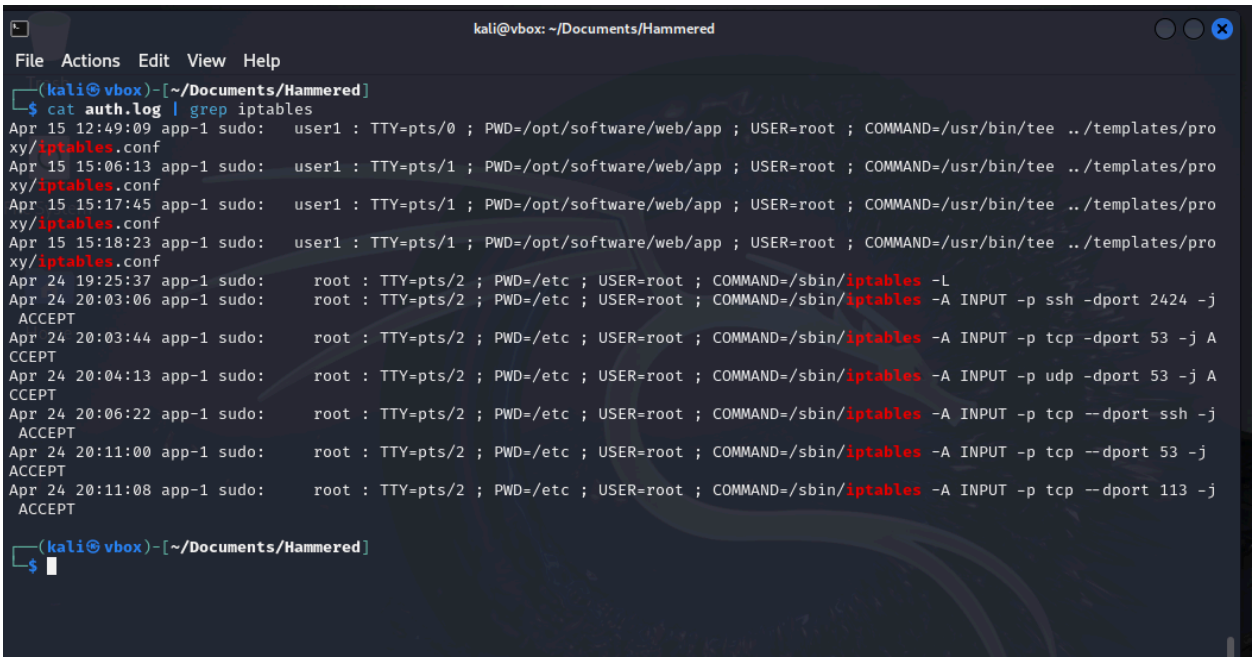

Figure 12: Output of auth.log containing iptables

From the output above shows firewall rules. Any line that has ‘-A INPUT…’ signifies that its a rule that allows traffic to a destination port.

There are 6 firewall rules.

Q8: One of the downloaded files to the target system is a scanning tool. Provide the tool name

The dpkg.log is a log file that contains records of packages (programs) installed, removed, upgraded and configured on a Debian Linux system. This log file is likely to contain an entry for a scanning tool. After running the cat command on the dpkg.log and browsing through the output, the tool nmap came up. To narrow it down, I used grep with the pattern nmap to confirm my findings.

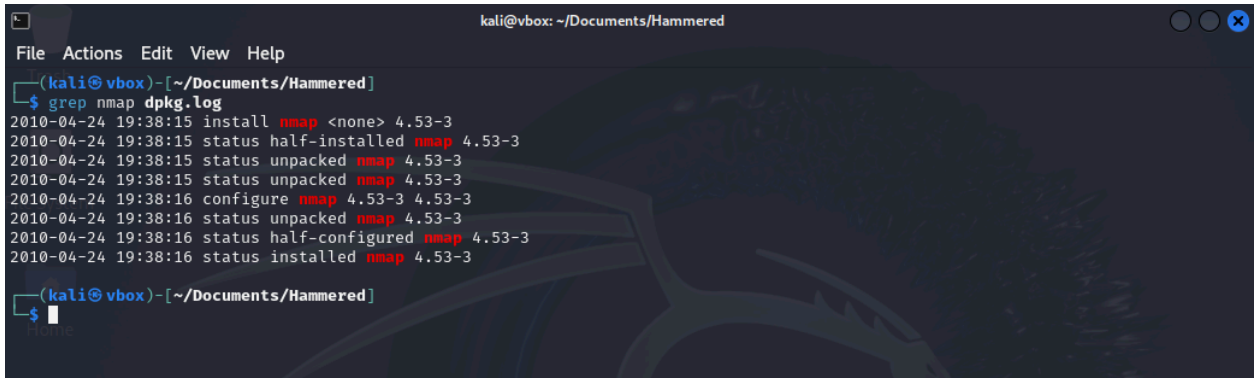

Figure 13: nmap entry in dpkg.log

Q9: When was the last login from the attacker with IP 219.150.161.20? Format: MM/DD/YYYY HH:MM:SS AM

Querying the auth.log and displaying lines that have successfully logged in with the IP address above.

cat auth.log | grep -i “accepted password” | grep “219.150.161.20”

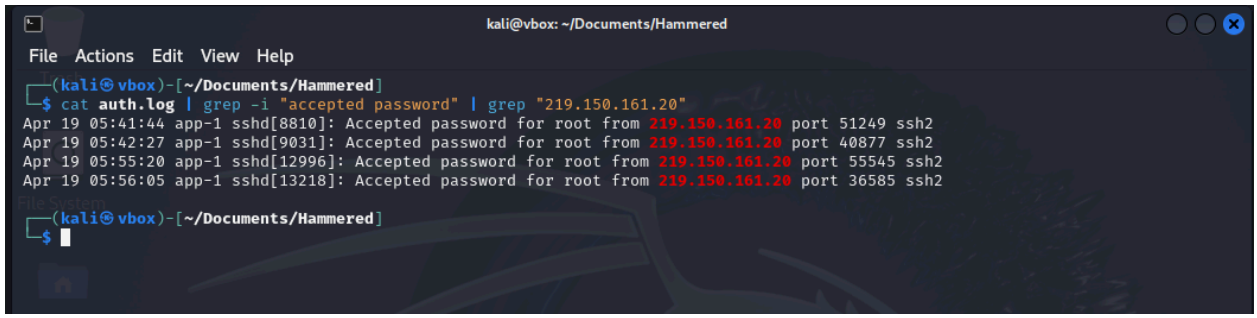

Figure 14: Entries in auth.log from the attacker

The output does not contain the year the entries were made. Can simply use the command ls -l to list the details of the file.

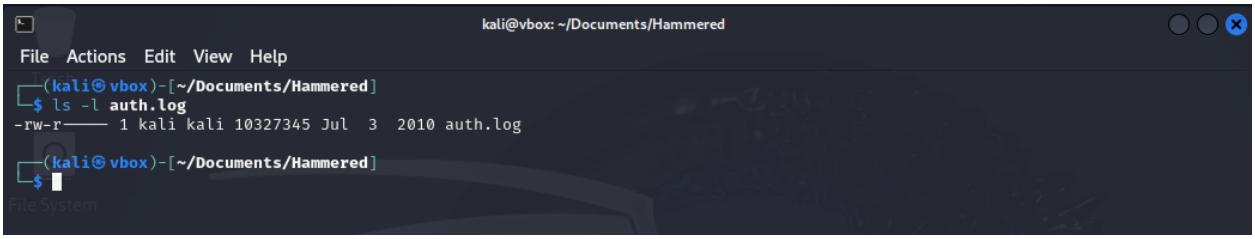

Figure 15: Details of the auth.log file

The file was created in the year 2010, therefore we can infer that the login entry was made on that year.

The attacker with IP 219.150.161.20 last login on 04/19/2010 05:56:05 AM

Q10: The database displayed two warning messages, provide the most important and dangerous one.

The daemon.log file records system service and background process events. If we output any lines from that file containing the word ‘warning’, can then determine which warning message is the most dangerous. Using the following command.

cat daemon.log | grep -i warning

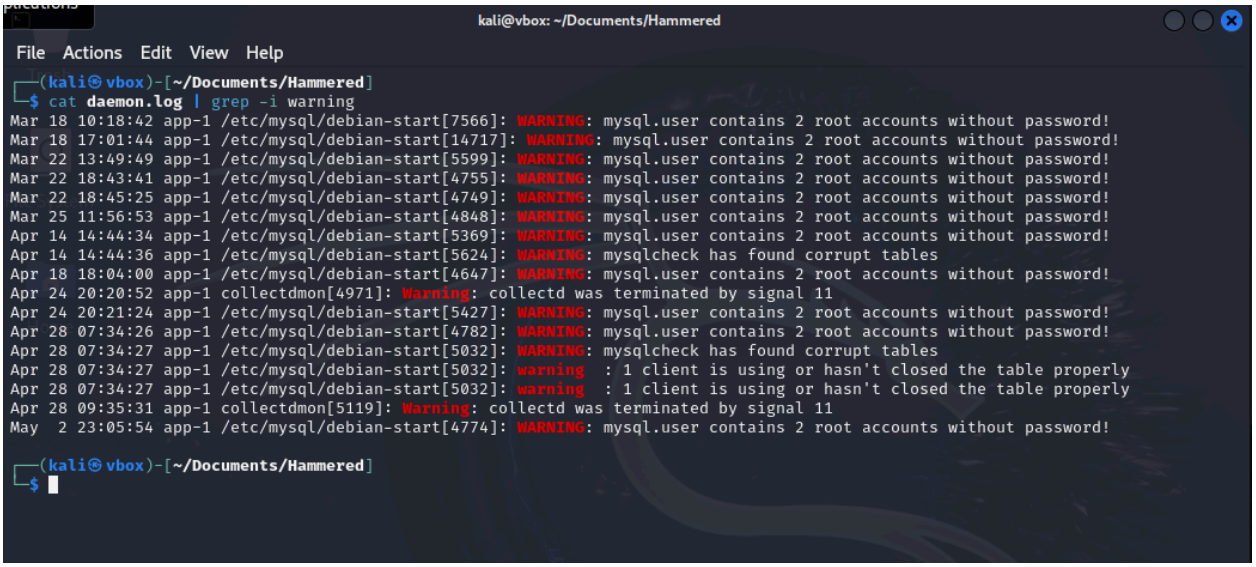

Figure 16: Entries containing waring from daemon.log

The output shows duplicate warnings. Filtering out the duplicates we have two warnings.

1: Warning: collected was terminated by signal 11

2: WARNING: mysql.user contains 2 root accounts without password!

We can infer that root accounts without passwords is the most dangerous warning.

Q11: Multiple accounts were created on the target system. Which one was created on Apr 26 04:43:15?

Querying the auth.log with the timestamp, we can discover which account was created then.

cat auth.log | grep 'Apr 26 04:43:15'

Figure 17: Shows account created on the given date

On April 26th 04:43:15, the account named 'wind3str0y' was created.

Q12: Few attackers were using proxy to run their scans. What is the corresponding user-agent used by this proxy?

For this question, we can look again at the file www-access.log. This file will contain user-agent information from incoming requests.

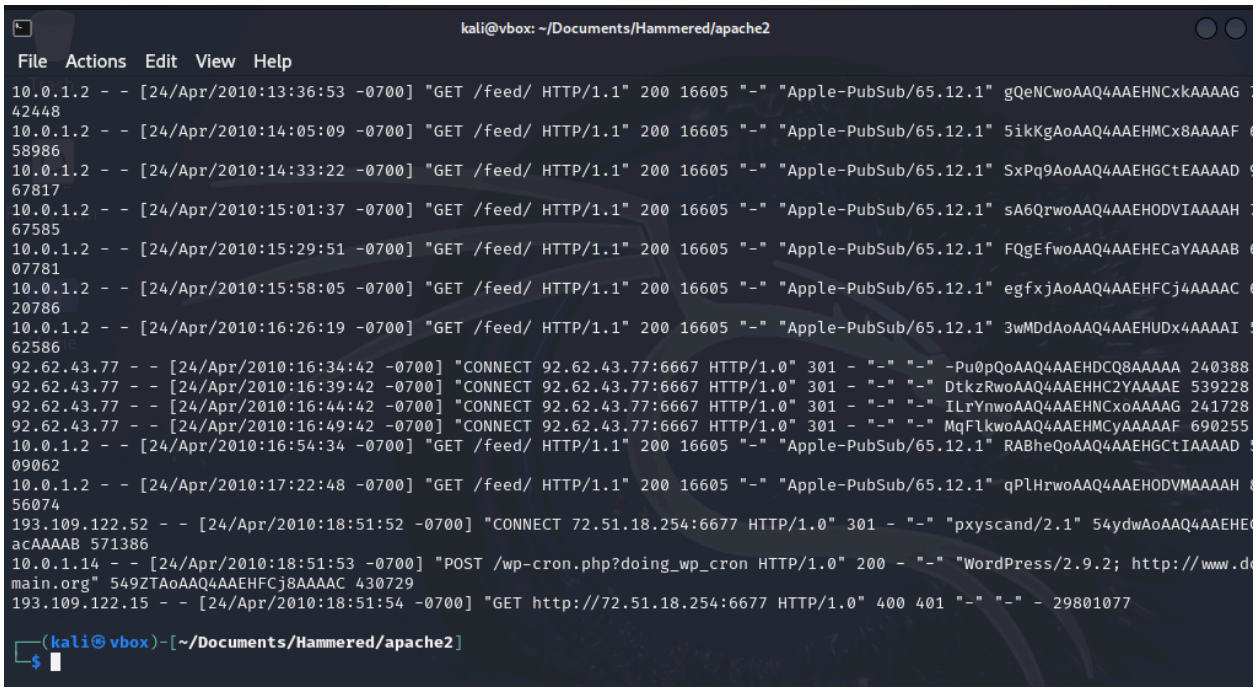

Figure 18: Output from www-acces.log

Browsing through the output, one line is of interest. The line 5th from bottom that reads:

193.109.122.52 - - [24/April/2010:18:51:52 -0700] “CONNECT 72.51.18.254:6677 HTTP/1.0” 301 - “-“ “pxyscand/2.1”

The string ‘pxyscand’ appears to be a program. Further investigation[2] pxyscand 2.1 is a network proxy scanner.

The scanning tool is pxyscand/2.1

[2] https://www.useragentstring.com/pxyscand2.1_id_17796.php

Conclusion

This lab involved investigation, inference, extensive use of Linux commands and working with log files to follow the trail of the attackers.

By delving into various logs I was able to improve my familiarity with the log formats and to identify which command would be best suited to retrieve the information required to answer the questions.

The investigation part of this lab was particularly interesting as it could represent a real world scenario. Trying too infer whether an attacker did attack the system by analyzing indicators of comprise from various logs.

Moving forward this lab solely focused on Detection phase in the incident response life cycle with relation to NIST SP 800-61 R3.

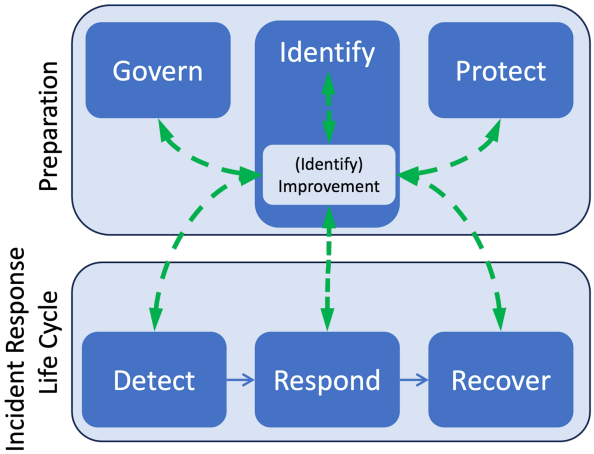

Figure 19: Incident Response Life Cycle[3]

Future improvements or to extend this lab could involve detailing the response and how the system would recover from such attacks.

[3] https://csrc.nist.gov/projects/incident-response